Semantic Analysis

Dragon: Ch 6. (Just part of it)

Holub:

Virtual Machine & Optimization Laboratory

Dept. of Electrical and Computer Engineering

Seoul National University

What is semantic analysis?

Semantic validity

- Parser and Lexer ensure the input has valid structure
- Need to check if the input has valid meaning (the meaning of a program is the result of the computation)

Static semantic checking at compile-time, for example,

- Type checking: if operand types match the operator
- Flow-of-control: if having well defined “jumps” (e.g., if there is a “continue”, there should be enclosing iterator)

Limitations of Semantic Analysis

Lexical & syntactic analysis is advanced

- Well worked-out theories to provide precise description of aspects of programming language
  - Regular expression and context free grammar
- It is possible to "compile" these descriptions into lexical and syntactic analyzer automatically
  - `lex` and `yacc`

Semantic analysis is less advanced

- A great deal of investigation has been going on in formal semantics of programming languages
- It is somewhat difficult to write a precise description of semantics of a programming language
- Automatic compilation of such a description into a semantic analyzer is beyond the state of the art
- There have been promising researches, but there still is a long way to go

Static and Dynamic Semantics

One issue is that the semantics of a program is not entirely determined at compile-time

In a typical programming language,

- Compiler decides some semantic issues (e.g., correct binding of types and names)
- Leave some to the binary code, so as to determine at run-time (e.g., out–of-bound array accesses)
  - However, compiler must assure that the semantics is preserved by the binary code
- The compile-time part: static semantics
  The run-time part: dynamic semantics

Ensuring Static Semantics

Semantic analysis phase deals with static semantics

- Should catch all compile-time semantic errors
- Keep track of types, declarations, scoping, etc for use in code generation

Declarations and Symbol Table

Declarations

- Associate "meaning" with names
- For example,
  - Variables (name, type, storage class, scope, etc.)
  - Functions (name, arguments, return type, external or not)
  - Types (type class, size, etc.) ….

Declarations and Symbol Table

To handle declarations, we use <u>symbol table</u>

- A global data structure to map a name with values or attributes at compile-time
  - E.g., `int x;` x is a variable whose type is integer
- Symbol tables become complicated when the mapping depends on contextual information (ex: block-structured languages like C)

There are two kinds of symbol table

- "Flat" symbol table
- "Scoped" symbol table

“Flat” Symbol Table

Symbol Table that depicts global name definition

- No contextual information
- A simple mapping from name to declaration
- Used in very simple compilers
  - Assemblers or macro processors

This is actually a dictionary abstract data type, which support:

- **Insert(Name, Decl)**: map the name with declaration
- **Lookup(Name)**: get the declaration with that name
- **enter(Name):** combination of the two (i.e., inserts Name if it is not already there and returns the declaration)

Flat Symbol Table in Lexer

Our project compiler will have two symbol tables, one of which is flat (i.e., the hash table we used in lexical analysis)

- Lexer enters the name of an identifier into this symbol table and returns declaration, which is a pointer to `struct id`

After lexical analysis, `struct id` pointers are used to represent identifiers everywhere

- Actually, they are the *names* in another, more complicated symbol table that handles scoping
- An advantage is that when we want to check if two identifiers are the same, we can compare their `struct id` pointers, instead of string comparison
  - We compare strings in the hash table, anyway, don't we?
  - Our implementation of the symbol table requires many comparisons

Data Structures for Flat Symbol Table

Linear search structures

- Array or linked List: easy to program, OK if list is short

Binary search tree

- Good asymptotic *log n* average performance
- In practice, not used in a compiler symbol table from an engineering viewpoint of programmability and performance

Hash table

- If there are many symbols, a hash table is good, and if implemented carefully, almost constant insert/lookup

Block Structure and Scoping

Block structure is one of the most useful language features

- Statement that can have its own data definitions that disappear after exiting the block
- Prevents accidental name clashes
- Ex: {*decls; stmts*} in C
- Blocks can be nested but cannot otherwise overlap

```
int x;     int y;
{
    float x;
    x += y;  /* float += int */
}
x += ..      /* int */
```

Scope and Extent in Block Structures

The "scope" of a declaration is the portion of a program text for which the declaration is "visible"

- Global declaration: entire program
- Local declaration: procedure or block
- Some names may have many scopes

The "extent" means the lifetime of the storage associated with the variable

- Scope and extent are usually the same
- Exception (e.g., static locals in C)

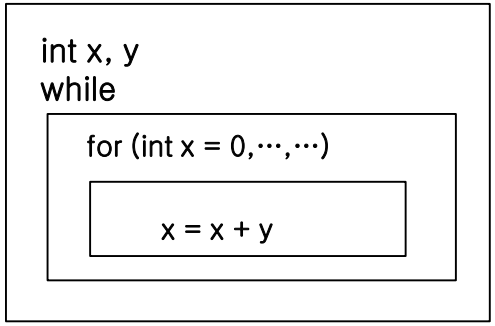
Scoped Name Definition

Compiler symbol tables are concerned exclusively with the scope, not the extent

- Bind names to attributes depending on the scope in which it occurs

Scope rule determines which declaration applies to a name instance: most-closely nested rule

- The scope of declaration in a block B includes B
- If name x is not declared in B, then an instance of x in B is in the scope of the declaration of x in the most closely enclosing block B'



“Scoped” Symbol Table

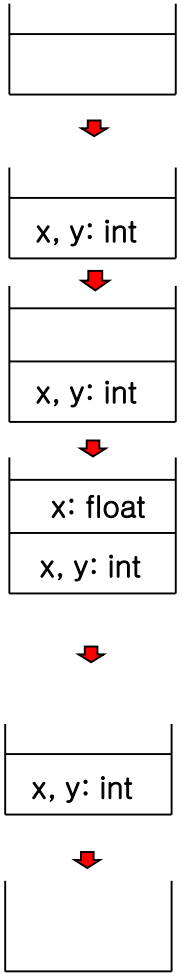
Abstract scoping operations: use stack paradigm

- **push_scope()**: start a new scope which becomes the “current scope”
- **pop_scope()**: return to the previous state; restore symbol table before the last **push_scope()**
- **insert(name, decl)**: basically the same, but it inserts the definition in the “current scope”
- **lookup(name)** must now search for the variable in all of pushed but not popped scopes in reverse order in which they were pushed; it returns the first definition

Example

```
{                          push_scope()
    int x;                 insert(x, var int decl)
    int y;                 insert(y, var int decl)
    {                      push_scope()
        float x;           insert(x, var float decl)

        x += y;            lookup(x) : float; lookup(y): int;
    }                      pop_scope()
    x += ..                lookup(x) : int
}                          pop_scope()
```



Implementation of Scoped Symbol Table

Stack of flat tables

- Implement a stack of array elements and each array element is a flat symbol table (e.g., hash table)
- **push_scope()** and **pop_scope()** literally push and pop a flat table
- **insert()** inserts in the current scope (table) and **lookup()** does a flat-table lookup in each element of the array from the top
- Problem: scopes with not many definitions either waste space or require complex implementation

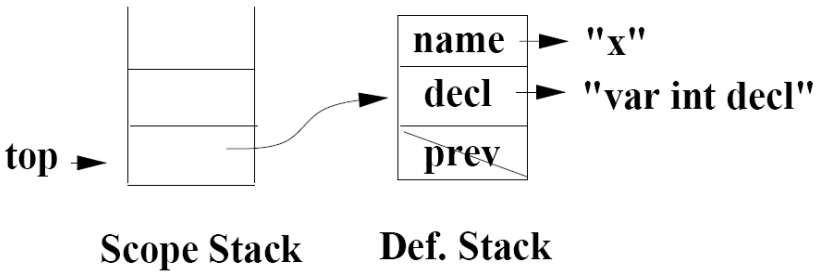
Our Implementation Choice

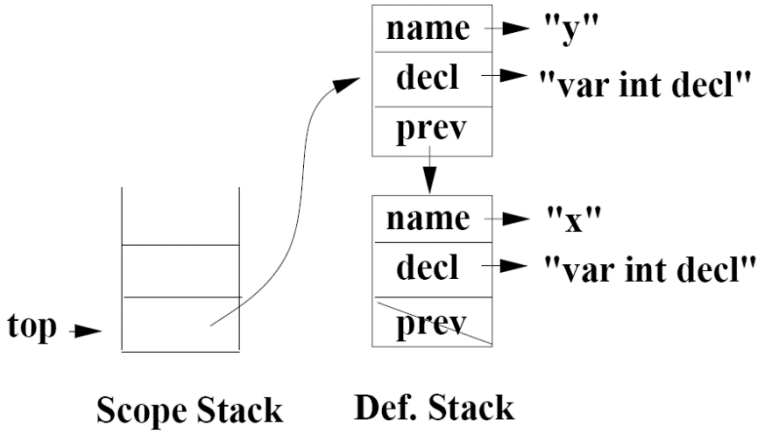
Stack of definitions

- Keep a stack of individual definitions (not scopes) and mark scope boundaries so that **pop_scope()** knows how many definitions to remove from the top of the stack
- Two methods to make the boundary
  - Inserts a pseudo definition that is recognized as a marker
  - Maintain a separate scope stack which points the to the top of stack when a scope was pushed: we take this approach
- **insert()** always inserts to the top of definition stack
- **lookup()** searches *backwards* in the table

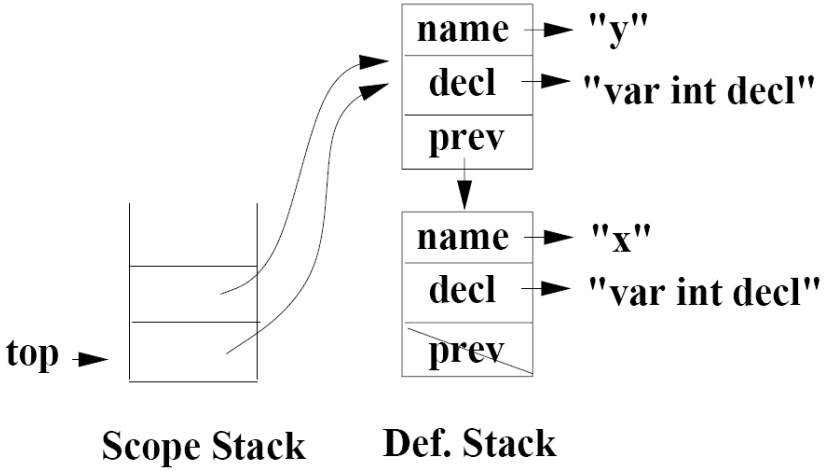
Example

After push _scope(), insert(x, var int decl) → After insert(y, var int decl)

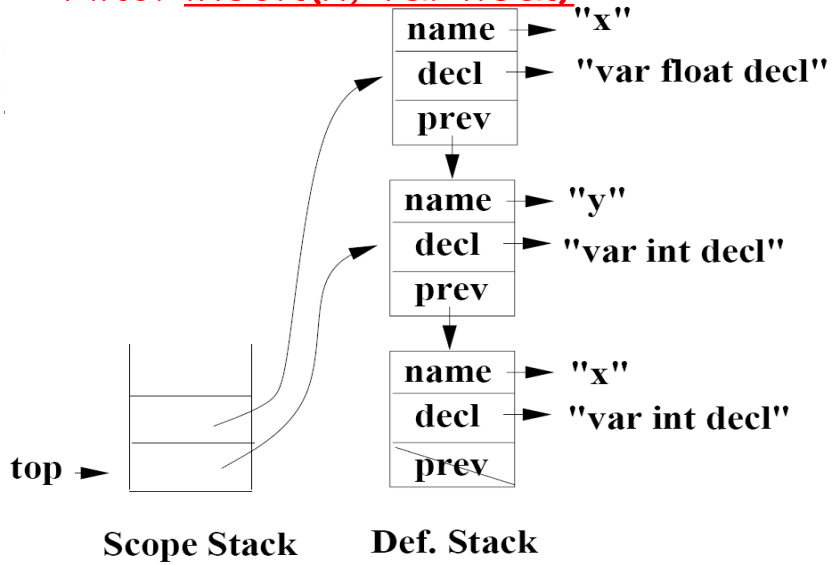




→ After push_scope() →



After insert(x, var float)



Another Implementation: Hashed Table

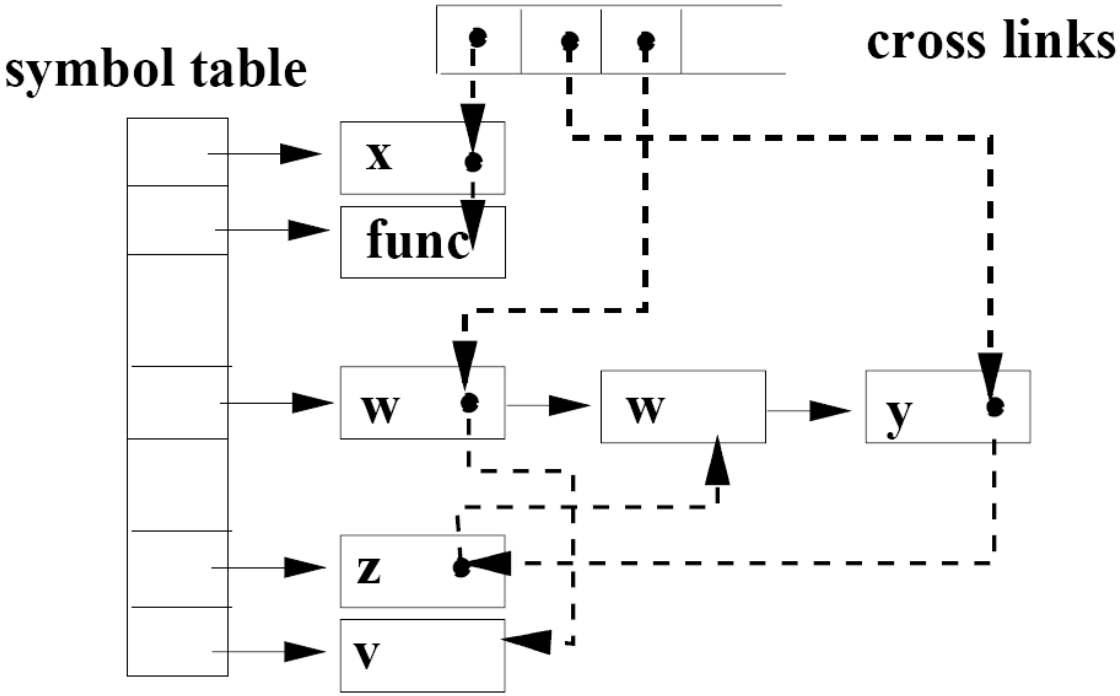
Maintain a single hash table that implements open hashing

A name is hashed and inserted at the beginning of the linked list for that hash slot

Example Hashed Symbol Table

Example:

```
int x;
func(int y, int z)
{
    int w;
    while(expr) {
        int w, v;
    }
}
```



Declarations

Name definitions associate "semantic something" with a name, which is a data structure representing the declaration

- Processing declaration depends on the language semantics
- Declarations and names are completely independent things and the only association occurs in the symbol table
- Association may change in the context and one name may be associated with many declarations

There are many ways to process declarations and build a symbol table, and we will present one way that is relatively simple and directly applicable to processing C declaration

An Example of *subc.h*

Data formats and structures in "subc.h"

- IDs, symbol table entries, and declarations

```
struct id {
    char            *name;
    int             lextype;
};

/* symbol table entry */
struct ste {
    struct id               *name;
    struct decl             *decl;
    struct ste              *prev;
};
```

```
struct decl {
    int             declclass;    /* DECL class: VAR, CONST, FUNC, TYPE  */
    struct decl     *type;        /* VAR, CONST: pointer to its TYPE decl  */
    int             value;        /* CONST: value of integer constant  */
    float           real_value;   /* CONST: value of float constant  */
    struct ste      *formals;     /* FUNC: pointer to formal argument list  */
    struct decl     *returntype;  /* FUNC: pointer to return TYPE decl  */
    int             typeclass;    /* TYPE: type class: INT, array, ptr, …  */
    struct decl     *elementvar   /* TYPE (array): ptr to element VAR decl  */
    int             num_index     /* TYPE (array): number of elements  */
    struct ste      *fieldlist    /* TYPE (struct): ptr to field list  */
    struct decl     *ptrto        /* TYPE (pointer): type of the pointer  */
    int             size          /* ALL: size in bytes  */
    struct ste      **scope;      /* VAR: scope when VAR declared  */
    struct decl     *next;        /* For list_of_variables declarations  */
};                                /* Or parameter check of function call  */
```

An Example Declaration in *subc.y*

```
%union yystacktype
{
    int             intval;
    double          floatval;
    char            *stringval;
    struct id       *idptr;
    struct decl     *declptr;
    struct ste      *steptr;
}
%type           <declptr>       type type_id var var_list ...
%nonassoc       <idptr>         ID
%nonassoc       <intval>        INTEGER-CONST
%nonassoc       <floatval>      FLOAT-CONST
%nonassoc       <stringval>     STRING-CONST
```

An Example *subc.l*

```
...
<norm>{ID} {
    yylval.idptr = enter(ID, yytext, yyleng);
    return (yylval.idptr → lextype); // ID
}

<norm>{INTEGER-CONST} {
    yylval.intval = (int) strtol(yytext, (char**) NULL, 10);
    return (INTEGER-CONST);
}

<norm>{FLOAT-CONST} {
    sscanf(yytext, "%lf", &yylval.floatval);
    return (FLOAT-CONST);
}
```

An Example of *init_type()*

```
init_type() {
    inttype        = maketypedecl(INT);
    floattype      = maketypedecl(FLOAT);
    voidtype       = maketypedecl(VOID);
    ..
    declare(enter(ID, “int”, 3), inttype);
    declare(enter(ID, “float”, 5), floattype);
    returnid = enter(ID, “*return”, 7);
    ..
}
```

In this example, type specifiers like `int` are regarded as a token ID instead of a token TYPE and the lexer will give idptr to yylval; later yacc will look through the linked list of the symbol table to determine the declaration that was inserted during the initialization

Example: Simple Variable Declarations

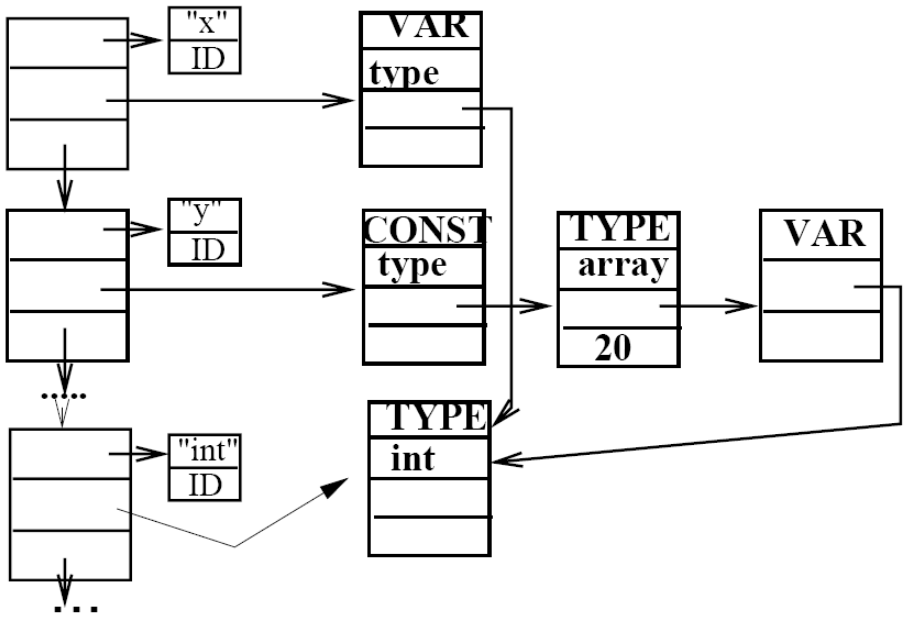
Example declaration

```
int y[20];
int x;
```

Grammar

```
var_decl : type ID ";"
         | type ID "[" const_expr "]" ";"

type     : type_id
         | ...
type_id  : ID
```



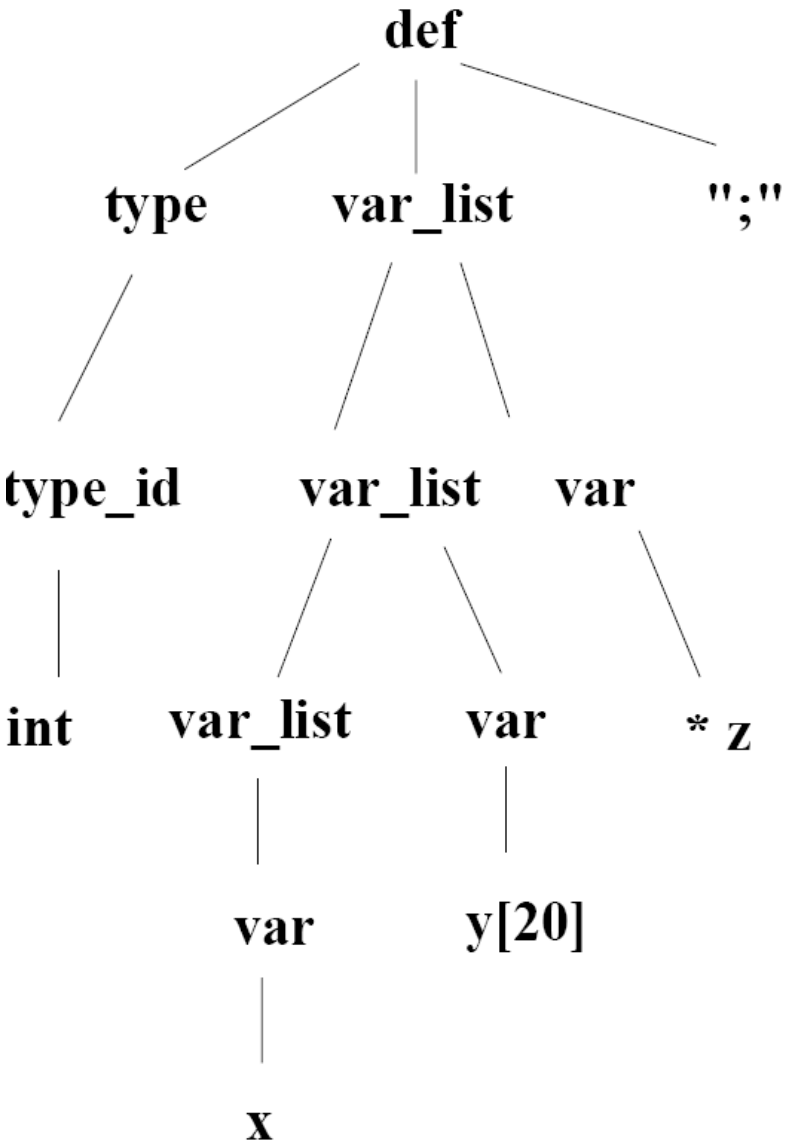
```
var_decl : type ID ";" { declare($2, makevardecl($1); }
         | type ID "[" const_expr "]" ";"
           {declare($2,makeconstdecl(makearraydecl($4,makevardecl($1))));}
type     : type_id {$$ = $1}
         | struct_specifier {$$=$1}
type_id  : ID
           { struct decl *declptr = findcurrentdecl($1);
             check_is_type(declptr);
             $$ = declptr;
           }
```

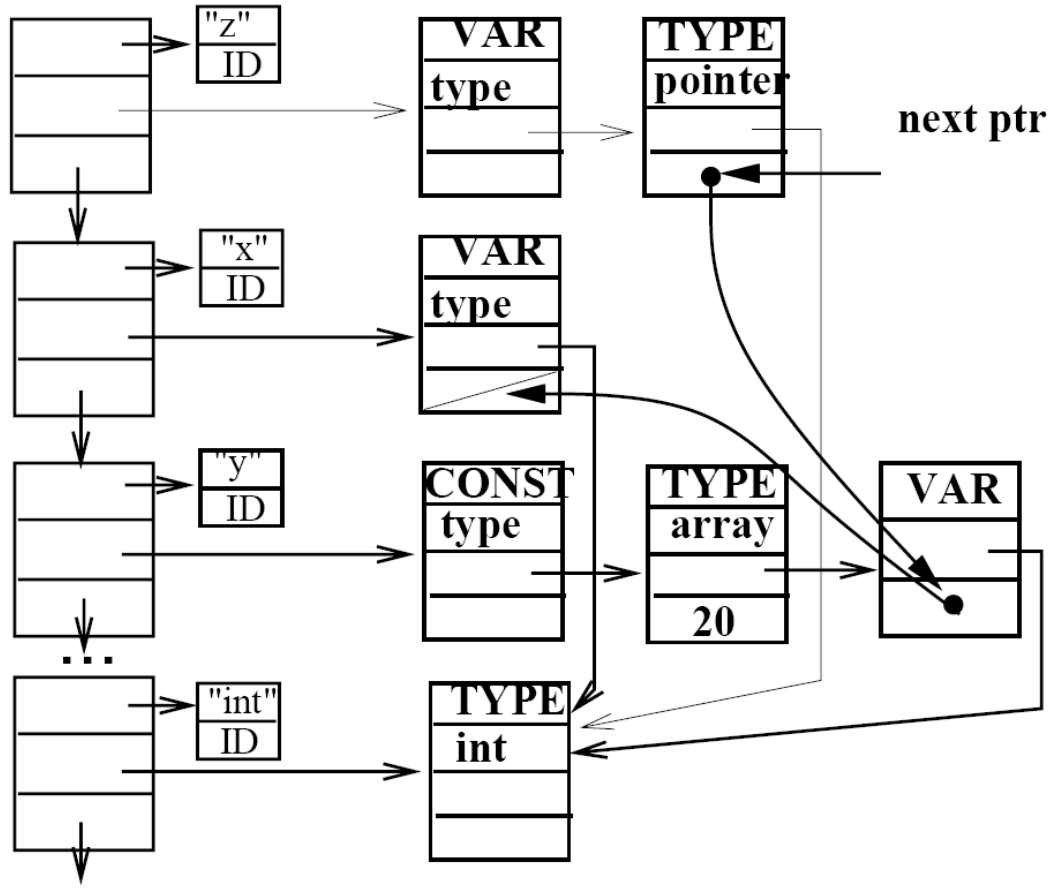
For array type decls, we made the elementvar ptr to point a VAR decl instead of TYPE decl, to make sure an element of the array in LHS of an assignment statement is a variable when we do the type checking

Example: List of Variable Declarations

Remember that `struct decl` has one more field, `next`, which links decls whose type is not yet defined

```
def       : type var_list ";" {add_type_to_var($1, $2);}
          ;
var_list  : var_list "," var   {$3→next = $1; $$ = $3}
          | var                {$$ = $1;} /* $1→next is assumed to be NULL */
          ;
var       : ID                          {declare($1, $$ = makevardecl(NULL));}
          | ID "[" const_expr "]" {declare($1,
                 makeconstdecl(makearraydecl($3, $$=makevardecl(NULL)));}
          | "*" ID             {declare($2, makevardecl($$=makeptrdecl(NULL)));}
          ;
```


def
type
var_list
";"
type_id
var_list
var
int
var_list
var
* z
var
y[20]
x

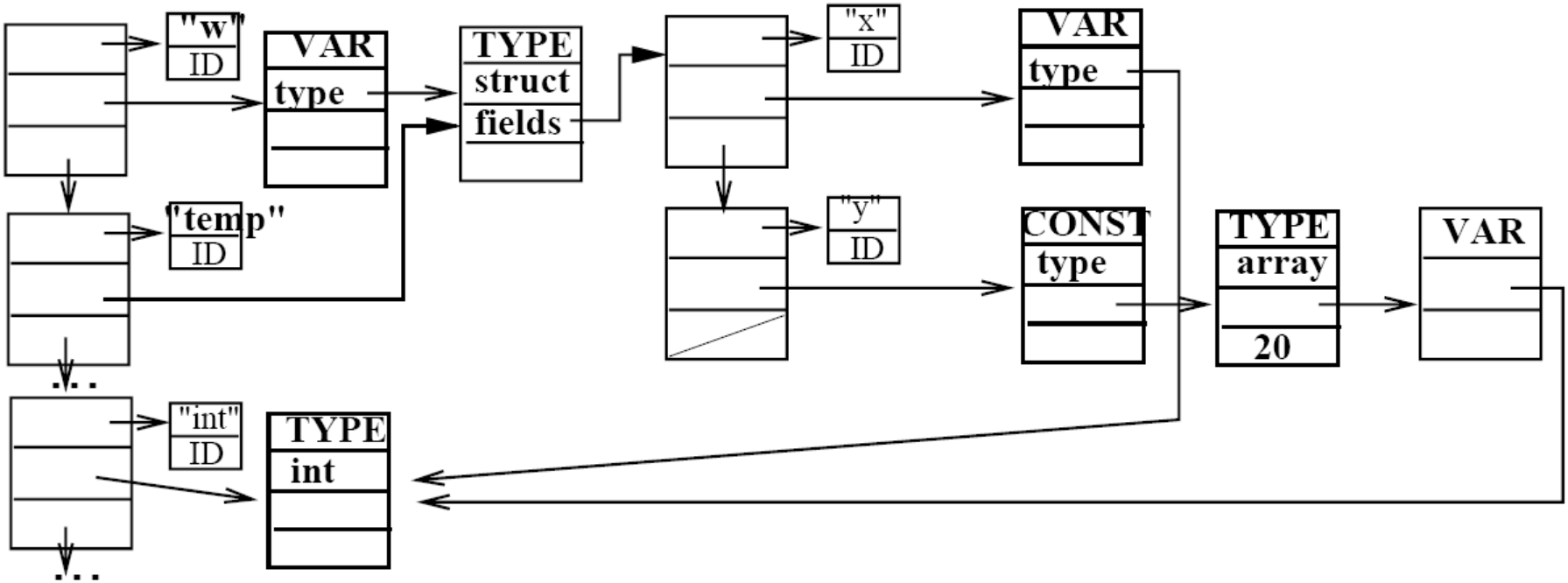

"z"
ID
VAR
type
TYPE
pointer
next ptr
"x"
ID
VAR
type
"y"
ID
CONST
type
TYPE
array
20
VAR
...
"int"
ID
TYPE
int

Example: Struct Declaration

Structures: how to collect fields?

```
struct_specifier : STRUCT tag "{"
                   { push_scope(); }
                     def_list /* popscope reverses stes */
                   { struct ste *fields = popscope();
                      declare($2, ($$=makestructdecl(fields))); }
                  "}"
                  | STRUCT tag
                   { struct decl *declptr = findcurrentdecl($2);
                      check_is_struct_type(declptr);
                      $$ = declptr;
                   }
                  ;
```

struct temp { int x; int y[20]; } w;



Examples: Function Declarations

```
func_decl: opt_type ID "("
            {
                struct decl *procdecl = makeprocdecl();
                declare($2, procdecl);
                pushscope(); /* for collecting formals */
                declare(returnid, $1);
                $<declptr>$ = procdecl;
            }
            var_list ")"
            {
                struct ste *formals;
                struct decl *procdecl = $<declptr>4;
                formals = popscope();
                /* popscope reverses stes (first one is the returnid) */
                procdecl→returntype = formals→decl; /* for type checking */
```

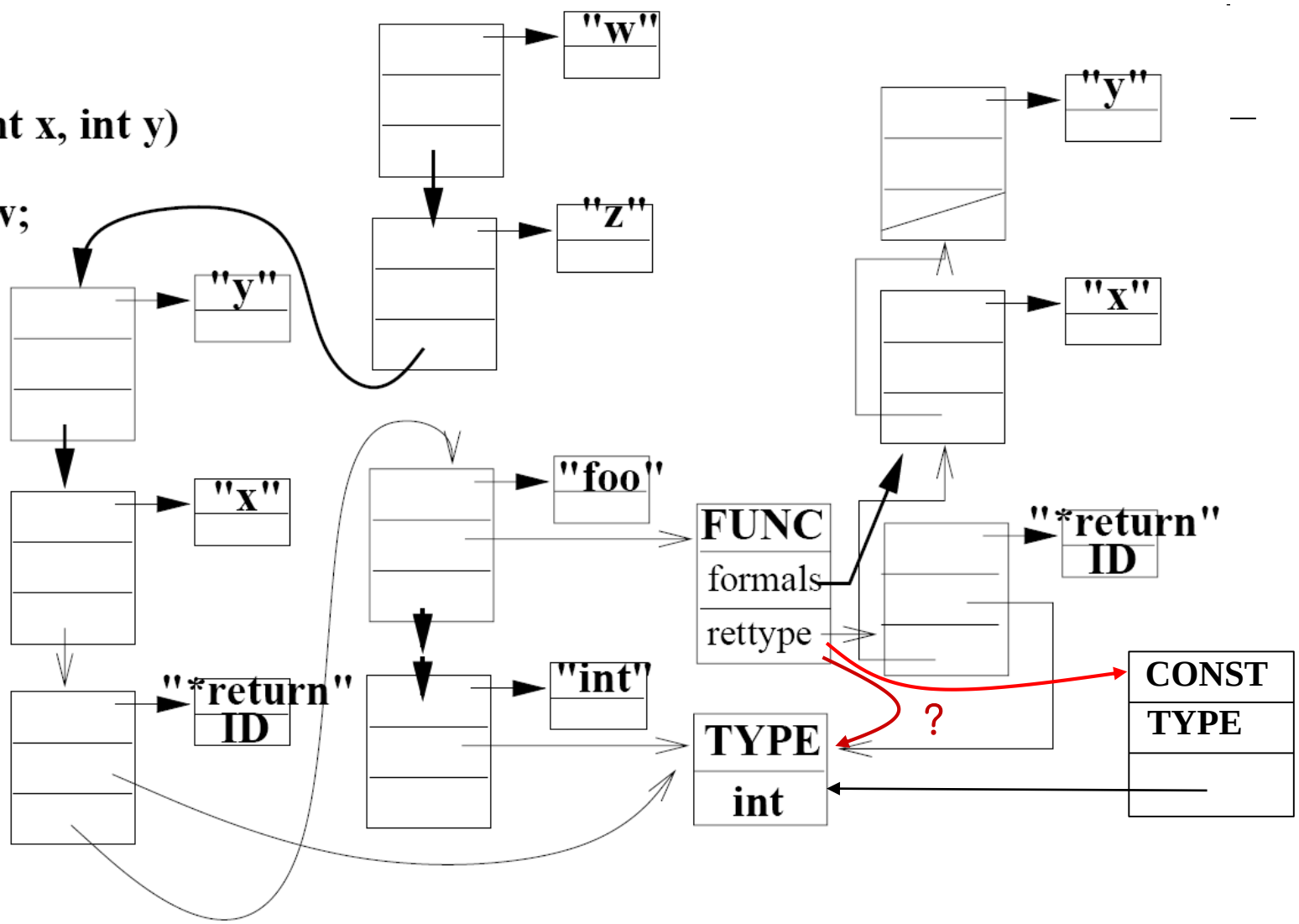
Examples: Function Declarations

```
            procdecl→formals = formals→prev;
            pushscope() /* for installing formals & locals in this scope */
            pushstelist(formals);
        }
        compound_stmts
        {
            popscope();
        }
opt_type: type_id           {$$ = $1;}
        | /* empty */       {$$ = inttype; }
        ;
```

For the type checking of return types within the function, we declare a fake ID *return in the symbol table and when we parse `return` *expr* ; we compare the current declaration of *expr* to the return type which can be get via findcurrentdecl(returnid)

```
int foo (int x, int y)
{
    int z, w;

}
```



stmt: RETURN expr; {checksametype(findcurrentdecl(returnid), $2);}

Some Type Checking Examples

```
unary          : INT-CONST   {$$ = makenumconstdecl(inttype, $1);}
               | ID          {$$ = findcurrentdecl($1);}
               | unary "." ID {$$ = structaccess($1, $3);}
               | unary "[" expr "]" {$$ = arrayaccess($1, $3);}
               ;
binary         | unary                {$$ = $1→type;}
               | binary '+' binary {$$ = plustype($1, $3);}
               ;
expr           : binary
               ;
assignment     | unary "=" expr {check_isvar($1);
                                 check_compatible($1, $3);
                                 $$ = $1 →type;}
               ;
```

When **unary** becomes **binary**, we take type information and propagate it

Array and Structure Accesses

```
struct decl *arrayaccess (struct decl *arrayptr, struct decl *indexptr) {
    struct decl *arraytype = arrayptr→type;
    check_isarray(arraytype);
    check_sametype(inttype, indexptr);
    return (arraytype→elementvar);
};

struct decl *structaccess (struct decl *structptr, struct id *fieldid) {
    struct decl *typeptr = structptr →type;
    check_isstruct(typeptr);
    return (finddecl(fieldid, typeptr →fields));
}
```

Example: Function Calls

```
unary    : unary "(" args ")"
           { checkisproc($1);
             $$ = checkfunctioncall($1, $3); }

args     : expr "," args { $1→next = $3; $$ = $1; } // Do not work correctly
         | expr          { $$ = $1; }
         ;

struct decl *plustype(struct decl type1, struct decl type2)
{
    struct decl *type_after;
    type_after = check_compatible_type(type1, type2);
    return (type_after);
}
```

```
struct decl * checkfunctioncall(struct decl *procptr, struct decl *actuals)
{
    struct ste *formals = procptr→formals;
    /* 1. compare the number of formals and actuals */
    /* 2. and check for type match                  */

    while(formals != NULL && actuals != NULL) {
        checkisvar(formals→decl);
        check_compatible(formals→decl, actuals);
        formals = formals→prev;
        actuals = actuals→next;
    }
    return (procptr →returntype); /* for decl of the call; do not work correctly*/
}
```

Above method of argument checking does not work for actuals

Type Theory

Why use types?

- Enforce rules for operations implicitly (e.g., can use “+” for adding integers)
- Reduce semantically incorrect computations (e.g., add integer and string)

Type system

- How to define types and associate them with language constructs
- Rules for type equivalence, type compatibility, and type determination

Type Equivalence: Which Types are Same?

Two type equivalence: Structural Equivalence and Name Equivalence

- Structural Equivalence: Same Type Expression
- Name Equivalence: Same Type Name
- Ex: `struct s1 { int a;};` and `struct s2 { int a; };`
  : structurally-equivalent but not name-equivalent

C allows (loose) name equivalence

```
typedef int farenheit; typedef int celcius;
farenheit x; celcius y;
x = y;  // type error? No
```

Comparing pointers to `struct decls` is not enough to decide type equivalence (because of type coercion; see slide 44), but helps to determine it quickly if they are equal

Type Compatibility: Which Types Can be Used?

Operand Compatibility

- What combinations of operators and operands are allowed by the language

Assignment Compatibility

- Check the correctness of assignment
- Function calls: the formals must be assignment compatible with the actuals

Type Determination: Decide Type of Expression

Simple model: type of an expression depends on its operands
Ex: int + int → int

- Literals (numbers or strings):
  - Lexical type determines its type
- ID: type depends on its declaration
- Compound expression: function of operator and operands

Type conversion: programmer explicitly request the compiler to insert code to convert one type to the other

```
int *x; x = (int *) malloc(4);
```

Type coercion: implicit type conversion that occurs during assignment or when evaluating an expression

```
int x; short y; float z;
y = x; z = x;
```

Type Checking

The process of verifying and enforcing the constraints of types to ensure that the program is *type-safe*, minimizing the chances of type errors

- e.g., x = “3” + 2; which is allowed by some language but not by others, throwing compile-time or run-time errors
- Type checking can be done statically (compile-time) or dynamically (run-time)
- Type checking can be strong or weak

Statically-typed vs. Dynamically-typed

Statically-typed languages

- Type of a variable is known at **compile-time** instead of at run-time, so type error can be checked statically (e.g., C/C++, Java, Fortran);
  - In Java, the following causes compile error: `int x; x = “Hello World”;`
  - Errors are checked earlier; computations can be efficient

Dynamically-typed languages

- Type of a variable is known and checked at run-time (e.g., Python, JavaScript, PHP);
  - In Python, the following is fine: `x = 10; x = “Hello World”;`
  - No need for compilation; exploit dynamic binding intensively

Strongly-typed vs. Weakly-typed

Strongly-typed languages

- Strongly-typed: enforces the type rules strongly, so type mismatches lead to type errors, regardless of when type checking occurs (e.g., Python, Java)
  - In Python, the following makes a run-time error: `x = “Hello World” + 10;`
- For those languages, the compiler or the run-time system guarantee that the program will run without type errors
  - In Python, the following makes a run-time error: `x = “Hello World” + 10;`

Weakly-typed languages

- Weakly-typed: variables are less bound to a specific type, so type safety is less enforced (e.g., C/C++, PHP, JavaScript)
  - In C, the following is fine: `int x; x = “Hello World”; printf(“%d”, x);`